Guía de estudio

Tarea 1.1: Cambiar el tipo de una instancia con AWS CLI

Pau López Núñez (PLN)

1. De qué va la tarea

En EC2 el **tipo de instancia** (t3.micro, t3.small...) decide el hardware de la máquina virtual: cuántas CPU, cuánta memoria y cuánta red. Cambiarlo sirve para ajustar el servidor a lo que necesita (rightsizing): más potencia si se queda corto, o menos para ahorrar.

AWS **no deja cambiar el tipo con la instancia encendida**. El proceso siempre es:

```
detener  ->  esperar a que esté parada  ->  cambiar el tipo  ->  arrancar  ->  esperar a que esté
en marcha
```

La tarea es automatizarlo con un script de bash **seguro**: que compruebe todo antes de parar nada.

2. Parte 2: el proceso a mano y «Para pensar»

- Con la instancia encendida, `modify-instance-attribute` da el error `IncorrectInstanceState` (la instancia no está en estado stopped).
- Con stop, wait, modify, start y wait sí funciona. Cada espera tarda unos **17 segundos**.

Respuestas de «Para pensar»

- **ID:** no cambia. Es la misma instancia, con el mismo disco y las mismas etiquetas.
- **IP pública:** sí cambia cada vez que se para y arranca (54.166.48.196, luego 13.221.82.49, luego 13.221.162.203). La IP pública automática se libera al parar y al arrancar te dan otra.
- **IP privada:** no cambia (172.31.95.12), porque va ligada a la interfaz de red (ENI), que no se borra.
- **Clientes que usan la IP pública:** dejarían de conectar, porque la IP antigua ya no es tuya. Se soluciona con una **IP elástica** (fija), un **nombre DNS** (Route 53) o un **balanceador de carga**.
- **Por qué no se puede en caliente:** el tipo es el hardware real (CPU, memoria, red). Normalmente hay que mover la máquina a otro servidor físico, y el sistema operativo detecta CPU y memoria al arrancar.

3. El script, bloque a bloque

Cada bloque muestra el trozo de código y debajo qué hace.

Cabecera y preparación (líneas 1-25)

```
#!/usr/bin/env bash
# cambia_tipo_ec2.sh - Cambia el tipo de una instancia EC2
# Autor/a: Pau López Núñez (PLN)
# Fecha:    21/10/2026
# Uso:      cambia_tipo_ec2.sh [-y] ID_INSTANCIA TIPO_NUEVO
# Ampliación: comprueba que el tipo nuevo admite la arquitectura de la instancia
set -uo pipefail
export AWS_PAGER=""               # sin paginador: el script no se detiene
# Solo se modifican las instancias con esta etiqueta Propietario
PROPIETARIO="PLN"
I="Reservations[0].Instances[0]"
# Tamaños que el Learner Lab permite arrancar
TAMANOS_PERMITIDOS="nano micro small medium large"
error() {                                 # error MENSAJE CODIGO
  echo "ERROR: $1" >&2
  exit "$2"
}
uso() {
  echo "Uso: $(basename "$0") [-y] ID_INSTANCIA TIPO_NUEVO" >&2
  exit 1
}
```

- `#!/usr/bin/env bash`: que se ejecute con bash.
- `set -uo pipefail`: `-u` da error si usas una variable que no existe (evita fallos por un nombre mal escrito); `pipefail` hace que, si falla un comando dentro de una tubería, falle la tubería entera.
- No lleva `set -e` a propósito: comprobamos cada comando nosotros para dar un mensaje claro y el código de salida que toca.
- `AWS_PAGER=""`: sin esto la CLI v2 envía la salida a `less` y el script se queda parado.
- `PROPIETARIO="PLN"`: solo se tocan instancias con esa etiqueta.
- `I="Reservations[0].Instances[0]"`: atajo para no repetir esa ruta en cada consulta.
- `error()`: muestra el mensaje por la salida de errores (>&2) y sale con el código que le pases. Así cada fallo es una sola línea.
- `uso()`: muestra cómo se usa y sale con el código 1.

Paso 1: opciones y parámetros (líneas 27-39)

```
# --- 1. Opciones y parámetros ------------------------------------------
CONFIRMAR=1
while getopts ":y" OPCION; do
  case $OPCION in
    y) CONFIRMAR=0 ;;
    *) uso ;;
  esac
done
shift $((OPTIND - 1))
[ $# -eq 2 ] || uso
ID=$1
TIPO_NUEVO=$2
```

- `getopts ":y"` lee las opciones. Si aparece `-y` pone `CONFIRMAR=0`. Cualquier otra (por ejemplo `-x`) va a `uso`: código 1.
- `shift $((OPTIND - 1))` quita las opciones ya leídas, para que `$1` y `$2` sean el ID y el tipo.
- `[ $# -eq 2 ] || uso`: si no hay exactamente 2 parámetros, código 1.

Paso 2: la instancia existe (líneas 41-46)

```
# --- 2. La instancia existe (una sola llamada a describe-instances) ------
DATOS=$(aws ec2 describe-instances --instance-ids "$ID" --output text \
  --query "$I.[State.Name,InstanceType,Architecture,Tags[?Key=='Propietario'].Value | [0],Tags[
?Key=='Name'].Value | [0]]" \
  2>/dev/null) \
  || error "La instancia $ID no existe o no es accesible" 2
IFS=$'\t' read -r ESTADO TIPO_ACTUAL ARQUITECTURA PROP NOMBRE <<< "$DATOS"
```

- **Una sola llamada** a `describe-instances`, como pide el enunciado. Con `--query` (JMESPath) saca estado, tipo, arquitectura, etiqueta Propietario y etiqueta Name.
- `Tags[?Key=='Propietario'].Value | [0]` filtra las etiquetas y se queda con el valor de Propietario.
- Si el ID no existe, AWS da error; `2>/dev/null` oculta el mensaje de la CLI y `|| error ...` 2 sale con el código 2.
- `IFS=$'\t' read -r ...` reparte la salida (separada por tabuladores) en 5 variables.

Paso 3: la instancia es mía (líneas 48-50)

```
# --- 3. La instancia es mía ---------------------------------------------
[ "$PROP" = "$PROPIETARIO" ] \
  || error "La instancia $ID no tiene la etiqueta Propietario=$PROPIETARIO (tiene: $PROP)" 3
```

Si la etiqueta Propietario no es PLN, código 3. Así no tocas por error el servidor de otro.

Paso 4: no está terminada (líneas 52-56)

```
# --- 4. No está terminada ni terminándose -------------------------------
case $ESTADO in
  terminated|shutting-down)
    error "La instancia $ID está en estado $ESTADO" 4 ;;
esac
```

Si el estado es `terminated` o `shutting-down`, código 4.

Paso 5: el tipo es distinto (líneas 58-62)

```
# --- 5. El tipo es distinto ---------------------------------------------
if [ "$TIPO_NUEVO" = "$TIPO_ACTUAL" ]; then
  echo "La instancia $ID ya es de tipo $TIPO_ACTUAL: no hay nada que hacer."
  exit 0
fi
```

Si ya tiene el tipo pedido, lo dice y sale con el código 0: no hay nada que hacer.

Paso 6: el tipo es válido (líneas 64-70)

```
# --- 6. El tipo existe y su tamaño está permitido -----------------------
TAMANO=${TIPO_NUEVO#*.}
[[ " $TAMANOS_PERMITIDOS " == *" $TAMANO "* ]] \
  || error "Tamaño '$TAMANO' no permitido en el Learner Lab ($TAMANOS_PERMITIDOS)" 5
ARQ_TIPO=$(aws ec2 describe-instance-types --instance-types "$TIPO_NUEVO" --output text \
  --query "InstanceTypes[0].ProcessorInfo.SupportedArchitectures" 2>/dev/null) \
  || error "El tipo $TIPO_NUEVO no existe" 5
```

- `${TIPO_NUEVO#*.}` borra todo hasta el primer punto: `t3a.small` se queda en `small`.
- Comprueba que el tamaño está en la lista permitida; si no, código 5. Es **clave**: el Learner Lab **termina** las instancias mayores que large al arrancarlas, y con un xlarge perderías la instancia.
- `describe-instance-types` comprueba que el tipo existe (si no, código 5) y devuelve las arquitecturas que admite, que usa la ampliación.

Ampliación: la arquitectura (líneas 72-75)

```
# --- Ampliación: el tipo admite la arquitectura de la instancia -------------
# (un t4g es ARM: con una AMI x86_64 la instancia no arrancaría)
grep -qw -- "$ARQUITECTURA" <<< "$ARQ_TIPO" \
  || error "El tipo $TIPO_NUEVO es para $ARQ_TIPO y la instancia es $ARQUITECTURA" 5
```

Se explica en el apartado 5.

Paso 7: confirmación (líneas 77-86)

```
# --- 7. Confirmación ---------------------------------------------------------
echo "Instancia:   $ID ($NOMBRE)"
echo "Estado:      $ESTADO"
echo "Tipo actual: $TIPO_ACTUAL"
echo "Tipo nuevo:  $TIPO_NUEVO"
if [ "$CONFIRMAR" -eq 1 ]; then
  [ "$ESTADO" = "running" ] && echo "AVISO: la instancia se detendrá y volverá a arrancar."
  read -r -p "¿Continuar? (s/n): " RESPUESTA
  [ "$RESPUESTA" = "s" ] || error "Cancelado por el usuario" 6
fi
```

Muestra un resumen. Sin `-y` avisa de que la instancia se va a detener y pregunta. Si no respondes exactamente `s`, sale con el código 6 sin haber tocado nada.

El cambio de tipo (líneas 88-111)

```
# --- Cambio de tipo ----------------------------------------------------------
if [ "$ESTADO" = "running" ]; then
  echo "Deteniendo la instancia..."
  aws ec2 stop-instances --instance-ids "$ID" >/dev/null \
    || error "No se ha podido detener la instancia" 7
fi
# Si estaba deteniéndose o se acaba de detener, esperamos a que pare del todo
if [ "$ESTADO" = "running" ] || [ "$ESTADO" = "stopping" ]; then
  aws ec2 wait instance-stopped --instance-ids "$ID" \
    || error "La instancia no ha llegado a detenerse" 7
fi
echo "Cambiando el tipo a $TIPO_NUEVO..."
aws ec2 modify-instance-attribute --instance-id "$ID" \
  --instance-type "Value=$TIPO_NUEVO" \
  || error "No se ha podido cambiar el tipo" 7
if [ "$ESTADO" = "running" ]; then
  echo "Arrancando la instancia..."
  aws ec2 start-instances --instance-ids "$ID" >/dev/null \
    || error "No se ha podido arrancar la instancia" 7
  aws ec2 wait instance-running --instance-ids "$ID" \
    || error "La instancia no ha llegado a arrancar" 7
fi
```

- Si estaba en marcha, la detiene y espera.
- Si estaba en `stopping` (deteniéndose), también espera. Si no, el cambio fallaría con `IncorrectInstanceState`.
- Cambia el tipo con `modify-instance-attribute`.
- **Solo la arranca si estaba en marcha.** Si estaba detenida, la deja detenida: el script no debe cambiar nada más que el tipo.
- Si falla cualquier comando de AWS, código 7.

Comprobación final (líneas 113-120)

```
# --- Comprobación final en AWS -----------------------------------------------
FINAL=$(aws ec2 describe-instances --instance-ids "$ID" --output text \
  --query "$I.[InstanceType,State.Name]") \
  || error "No se ha podido consultar el estado final" 7
read -r TIPO_FINAL ESTADO_FINAL <<< "$FINAL"
echo "Resultado:   tipo $TIPO_FINAL, estado $ESTADO_FINAL"
[ "$TIPO_FINAL" = "$TIPO_NUEVO" ] || error "El tipo final no coincide con el pedido" 7
echo "Tiempo total: ${SECONDS} s"
```

- Vuelve a **preguntar a AWS** el tipo y el estado reales. No vale repetir lo que pediste: hay que comprobar que el cambio se ha hecho.
- Si el tipo final no coincide, código 7.
- `$SECONDS` es una variable de bash con los segundos que lleva ejecutándose el script: el tiempo total.

4. Códigos de salida

Código	Significado
0	Cambio realizado, o nada que hacer (el tipo ya era el pedido)
1	Uso incorrecto: número de parámetros u opción desconocida
2	La instancia no existe o no es accesible
3	La instancia no tiene tu etiqueta Propietario
4	La instancia está terminada o terminándose
5	Tipo no válido: no existe, tamaño no permitido o arquitectura distinta
6	Cancelado por el usuario en la confirmación
7	Error de AWS durante el cambio

5. Las pruebas y qué demuestra cada una

Prueba	Resultado	Qué demuestra
P1 sin parámetros	1	Controla que el uso sea correcto
P2 ID falso	2	Detecta instancias que no existen
P3 t3.xlarge	5	Rechaza tamaños peligrosos sin parar la instancia (después sigue running)
P4 respondes n	6	La confirmación funciona y no se toca nada
P5 t3.small en marcha	0	El caso normal: para, cambia y vuelve a arrancar
P6 -y con la instancia parada	0	Sin confirmación y la deja parada
E1 Propietario=OTRO	3	No toca instancias ajenas
E2 t3z.small	5	Un tipo con buen formato pero inexistente
E3 el mismo tipo	0	Nada que hacer
E4 -x	1	Opción desconocida
X1 t4g.small	5	Ampliación: arquitectura distinta, sin parar la instancia
X2 t3a.small	0	Ampliación: misma arquitectura, el cambio se hace

6. La ampliación: la arquitectura

Hay procesadores **x86_64** (Intel o AMD) y **arm64** (Graviton, los tipos con g: t4g, m7g...). Tu instancia usa una AMI de Ubuntu para x86_64, que **no arranca** en ARM.

El script original dejaba pasar `t4g.small` (el tamaño es válido y el tipo existe): paraba la instancia, el cambio fallaba y el servidor se quedaba parado. La solución:

- Pide la `Architecture` de la instancia en la misma llamada de antes (sigue siendo una sola).
- Pide las arquitecturas que admite el tipo nuevo (`ProcessorInfo.SupportedArchitectures`).
- `grep -qw` busca la palabra exacta (`-w`) sin mostrar nada (`-q`). Si no la encuentra, código 5 **antes de parar nada**.

X1 lo demuestra con t4g.small (código 5 y la instancia sigue igual). X2 es la prueba de control: t3a.small es AMD pero también x86_64, así que pasa la comprobación y el cambio se hace.

7. Preguntas que te puede hacer el profesor

¿Por qué no usas set -e?

Para controlar cada error yo mismo y devolver el código que pide la tabla.

¿Para qué sirve AWS_PAGER=""?

Para que la CLI no abra less y deje el script esperando.

¿Por qué compruebas el tamaño antes de parar?

Porque el Learner Lab termina las instancias mayores que large. Hay que fallar antes de tocar nada.

¿Qué hace ${TIPO#*.}?

Borra todo hasta el primer punto: t3.small se queda en small.

¿Por qué no arrancas la instancia si estaba parada?

Porque el script solo debe cambiar el tipo, no el estado.

¿Por qué consultas AWS al final?

Para comprobar que el cambio se ha hecho de verdad, no suponerlo.

¿Por qué una sola llamada a describe-instances?

Lo pide el enunciado y es más eficiente: con --query saco todos los datos de golpe.

¿Qué pasa con la IP pública al cambiar el tipo?

Cambia, porque la instancia se para y arranca. Para mantenerla, una IP elástica.

¿Qué es IncorrectInstanceState?

El error que da AWS si intentas cambiar el tipo sin que la instancia esté del todo detenida. Por eso se espera con wait instance-stopped.